Highlights

Bulk Loading in Extensible Index Frameworks

- Bulk loading is posed at the level of an extensible index framework: one optional support function suffices to make it possible.
- Overlap of non-leaf keys does not compound with tree depth; each construction method converges to a steady-state value.
- Packing pages by occupancy doubles the cost of a selective query; delegating split points to the operator class removes 87–89% of it.
- The space-filling order and the split rule interact asymmetrically: on real data the order matters only while the split rule is naive.
- Measured on 138 million OpenStreetMap points against an implementation shipped in PostgreSQL since release 14.

Bulk Loading in Extensible Index Frameworks

Abstract

Bulk loading a tree index by sorting the input and packing pages is a technique known since the 1980s, and packing algorithms for R-trees are textbook material. All of them, however, are formulated for a particular key type and rely on its geometric properties. Extensible index frameworks—of which the generalized search tree (GiST) is the most widely deployed instance—expose no such properties: an operator class defines containment, union, penalty and split, and nothing else. Consequently, indexes built on these frameworks have been constructed by repeated insertion, at $O(N \log_f N)$ random page accesses and with pages filled to roughly 70 %.

This paper studies bulk loading at the level of the framework: what an extensible index framework must ask of an operator class for sorting-based construction to be possible, and what it must do with the answer so that the resulting tree is no worse than one built by insertion. The first part is a single additional support function returning a comparator. The second is the substantive one: packing pages by occupancy systematically inflates the overlap of non-leaf keys. Each partitioning strategy converges to a steady-state overlap ω_∞—the expected number of nodes per level whose key matches a point query—reached one or two levels below the root and stable thereafter, so that the penalty is a constant factor at every step of the descent rather than a product over levels. Occupancy-based packing yields $\omega_\infty \approx 4$ against ≈ 1.05 for insertion; the index is built 6.8–8.0 times faster and answers point queries 2.4–2.5 times more expensively. Delegating the choice of split points back to the operator class over a multi-page buffer removes 87–89 % of the penalty while retaining a 3.0–3.5-fold speedup of the build.

The design is implemented in PostgreSQL, has shipped since release 14 and has been the default strategy for a family of operator classes since release 18. It is evaluated on uniform and clustered synthetic data and on 138 million OpenStreetMap coordinates, against insertion-based construction, occupancy-based sorted construction and Hilbert-order packing, together with a five-year deployment history including the one case in which the correctness condition of the method was violated in production. Two findings were not anticipated: the order and the split rule interact asymmetrically, an elaborate space-filling curve helping only while the split rule is naive; and on the real dataset insertion is *no better* than occupancy-based packing,

reversing the ordering seen on uniform data and suggesting that the advantage of local insertion does not survive skew.

Keywords: index construction, bulk loading, extensible index framework, generalized search tree, space-filling curve, R-tree, PostgreSQL

---

1. Introduction

Database systems have long stopped treating multidimensional data as a specialty. Geometric and geographic objects, ranges, full-text signatures and vector representations are handled by general-purpose systems alongside scalar types. No single tree structure serves all of them, and systems have responded with frameworks rather than structures: the generalized search tree [1] implements page layout, descent, split, the concurrency protocol [2] and write-ahead logging, and delegates the data-specific decisions to an *operator class*. Substituting bounding rectangles yields an R-tree [3]; substituting signatures yields a full-text index; substituting ranges yields a range index. This is what makes an index framework extensible by users rather than only by the authors of the system [4, 5].

Extensibility is bought with ignorance. The framework knows containment (`consistent`), key combination (`union`), the cost of extending a key (`penalty`) and node partitioning (`picksplit`), and nothing else — in particular, no order on keys, and no way to derive one. Bulk loading, which for a B-tree means sorting the input and packing pages, is therefore unavailable, and indexes on such frameworks are built the only way that requires no order: by inserting entries one at a time. Every insertion descends to a leaf, so construction costs $O(N \log_f N)$ page accesses; the accesses are random, because insertion order is unrelated to physical placement; and pages end up filled to about 70 %, because they are produced by splits rather than by packing.

The gap is conspicuous because bulk loading of R-trees is well understood. Packed R-trees [6], Hilbert packing [7] and sort-tile-recursive packing [8] all sort by some function of the key and fill nodes consecutively. None of them applies here: each is formulated in terms of coordinates or of the area of a bounding rectangle, and an operator class over signatures or over ranges offers neither.

Problem.. We therefore ask the question one level up. Given an extensible index framework, (i) what is the minimal additional information the framework must obtain from an operator class for sorting-based construction to be possible, and (ii) what must the framework do with that information so that the resulting tree is no worse, for queries, than one built by insertion?

The answer to (i) turns out to be small: one optional support function returning a comparator over keys. The order need not be total in any meaningful sense, and

it need not be derivable from the query semantics; it must merely preserve locality, and only the operator class can know which order does.

The answer to (ii) is the substance of this paper, because the obvious procedure is wrong in a way that measurement of construction time cannot reveal. Filling pages to capacity and starting a new page chooses split points by a single criterion — occupancy. In one dimension that criterion is optimal. In more than one it is not: an order that preserves locality does so only approximately, and a page boundary that falls inside a dense region produces two neighbouring pages whose keys overlap substantially. Overlap determines the cost of search in a generalized tree: a query descends into every subtree whose key matches it. Measured per level, the overlap of an index built this way settles at about four — four subtrees entered at each level instead of one — and stays there as the tree grows deeper, whereas partitioning by the operator class settles at about 1.3. The index is built several times faster, occupies less space, and answers point queries twice as expensively, consistently across an order of magnitude in data volume. A comparison of construction methods by construction time — the comparison such papers customarily report — hides this entirely.

Contributions..

1. We formulate bulk loading as a framework-level problem and show that a single comparator-returning support function is sufficient, so that the technique becomes available to an arbitrary operator class rather than to one key type (Section 5). Changing the space-filling order requires writing one function and touches neither the tree, nor logging, nor concurrency: we demonstrate this by supplying Hilbert order as a drop-in alternative to Z-order, an operator class differing from the built-in one in exactly one support function out of ten (Section 8).
2. We show that the order and the choice of split points are two separate levers, both of which live in the operator class, and we measure their interaction. On synthetic data they are partially interchangeable: a better curve recovers 44 % of the penalty of occupancy-based packing, a better split rule recovers 53 %, and together they bring query cost within 4 % of insertion-based construction. On 138M real points the relation is asymmetric: the curve recovers 33 % when split points are chosen by occupancy and nothing at all when they are chosen by the operator class. Investing in an elaborate space-filling order therefore pays off only while the partitioning rule stays naive (Section 8).
3. We characterise the overlap introduced by occupancy-based packing. It does not compound with depth, as one might expect from the fact that a node is visited only if its parent was; instead each strategy converges to a steady-

state value ω_∞ within one or two levels of the root. We measure ω_∞ for both strategies, across data volume and degree of clustering (Section 6).

4. We give an algorithm that delegates split point selection back to the operator class over a buffer of k pages, removing 87–89 % of the penalty, and analyse the dependence on k (Section 6). The residual is the price of choosing split points within a buffer rather than globally, and we quantify it against the insertion-built baseline.
5. We report five years of deployment: the design ships in PostgreSQL since release 14 and is the default for a family of operator classes since release 18. We include the one case in which the correctness condition of the method was violated and what it implies for framework design (Section 9).

2. Background

A generalized search tree is a balanced tree whose node occupies one page. Each entry of an internal node is a pair (key, downlink), and the key covers the keys of all entries in the referenced subtree. A search descends into every subtree whose key may contain an answer, so unlike in a B-tree several descent paths may be active simultaneously; the cost of a search is governed by how many they are.

An operator class supplies the data-specific operations. In PostgreSQL these are `consistent` (may this subtree contain an answer?), `union` (a key covering a set of keys), `penalty` (the cost of extending a key by an entry), `picksplit` (partition an overflowing node in two), `same`, the optional `compress`, `decompress` and `fetch`, and the optional `distance` for nearest-neighbour search. The first four determine the shape of the tree completely: substituting bounding rectangles yields an R-tree, substituting signatures yields a full-text index, substituting ranges yields a range index.

Two properties of this interface matter throughout the paper. First, it contains no ordering operation, and none of the operations induces one. Second, `picksplit` encodes the operator class author's own notion of a good partition. We will use exactly that notion in Section 6 rather than inventing a criterion of our own; this is what keeps the construction algorithm framework-level rather than specific to a key type.

Concurrency follows Kornacker [2]: pages of a level are chained by right links, and a split is arranged so that a search arriving at a page that has been split can still reach the data by following the right link, without holding a lock on the parent. A detailed account of the implementation is given in [5].

3. Related work

Sort-based packing of R-trees.. Roussopoulos and Leifker [6] build a packed R-tree by sorting rectangles on a coordinate and filling nodes consecutively. Kamel and Faloutsos [7] sort on the Hilbert value of the centre, obtaining better locality. Leutenegger et al. [8] propose sort-tile-recursive packing, which partitions space into slabs; García et al. [9] choose the partition greedily; Böhm and Kriegel [10] address the high-dimensional case. Achakeev et al. [11] make the packing adaptive to an expected query profile and later parallelise it [12]; recent work learns the partitioning policy [13]. All of this line is formulated in terms of coordinates, of the area of a bounding rectangle, or of a query profile over such objects. An operator class over signatures or over ranges provides none of these, so none of the algorithms transfers to the framework setting as stated.

Generic bulk loading.. The closest prior work is van den Bercken et al. [14], who give a bulk loading algorithm explicitly called generic: it applies to a wide range of index structures, level by level, using Arge's buffer tree to batch insertions. Arge et al. [15] extend the approach from loading to bulk updates and queries, and the Priority R-tree [16] attains worst-case optimal query bounds.

The distinction from the present work is worth stating precisely, because the word "generic" is the same and the meaning is not. Buffer-tree loading is generic over structures that have an insertion algorithm: it batches insertions so that each of them is cheap in expectation, and it therefore requires no order on keys, which is exactly why it is applicable so widely. What it produces is the tree that insertion would have produced, at lower I/O cost: pages are filled by splits, and the shape is the shape insertion gives. Sorted packing requires more from the operator class—an order preserving locality—and gives more in return: pages are filled to the configured factor rather than by splitting, and the shape is determined by the order and by the choice of split points rather than by the history of insertions. The two are complementary, and neither dominates: buffer-tree loading is the method of choice when no locality-preserving order exists, which for some operator classes is the case.

Our contribution is therefore not bulk loading that is generic in the sense of requiring nothing from the structure, but the identification of the minimal thing a framework must *require* for the sorted variant to be available at all—and of what it must then do so that the result is not worse than what insertion would have given.

Split quality.. The R*-tree [17] improves the shape of the tree by accounting for perimeter and by forced reinsertion; the RR*-tree [18] refines subtree choice on insertion. These optimise the tree built by insertion. An attempt to improve the

penalty function of a generalized search tree along the same lines [19] proved inapplicable in a production system: changing the penalty function changes the shape of indexes built by earlier versions, and a database system must keep existing indexes usable after an upgrade. Sorted construction is free of that constraint, because it affects only newly built indexes—a point worth recording, since it explains why the same idea succeeds in one form and fails in another.

Extensible index frameworks.. The generalized search tree [1] is the framework studied here; concurrency and recovery for it are due to Kornacker et al. [2]. A parallel line covers space-partitioning structures: SP-GiST and its realization in PostgreSQL [20] raise the same class of question—what the framework must ask of the extension—for a different family of trees. The extensible index machinery of PostgreSQL as it stands is documented in [5].

Surveys.. Gaede and Günther [21] survey multidimensional access methods; Graefe [22, 23] surveys B-tree techniques, including bulk construction, in the one-dimensional setting where an order is given rather than requested. Comer's earlier survey [24] and the original B-tree paper [25] fix the terminology we use for the one-dimensional case.

Page-level optimisation.. An orthogonal way to speed up insertion-based construction is to make the individual page update cheaper [26]. Intra-page indexing [27] attacks the CPU cost of scanning a node; Section 8 shows why that cost matters. Both are complementary to this work.

External-memory sorting.. The cost of the sorting step is that of external merge sort in the standard I/O model [28].

4. Cost model

Let N be the number of entries, B the page size, M the memory available for sorting, f the node fan-out, φ the page fill factor, H the tree height, and c_r, c_s the cost of a random and of a sequential page access.

Construction by insertion descends to a leaf for every entry:

$$P_{\text{ins}} = N\,(H + \gamma)\,c_r, \tag{1}$$

where γ is the average number of extra accesses caused by splits and by updates of parent keys. Construction by sorting costs an external sort followed by a single sequential pass writing the packed levels:

$$P_{\text{sort}} = \frac{2N}{B} \log_{M/B} \frac{N}{B}\, c_s + \frac{N}{f\varphi}\, c_s. \tag{2}$$

Three sources of gain are visible: random accesses become sequential, splits disappear, and φ rises from the ≈ 0.7 characteristic of split-produced pages to the configured fill factor. The last also shrinks the index, and hence the cost of every later full scan of it.

The comparison is incomplete, because it accounts only for construction. Define the *overlap* of level l as the expected number of level-l nodes whose key matches a random point query,

$$\omega_l = \mathbb{E}\big|\{\, u \in U_l : \texttt{consistent}(\mathit{key}(u), Q)\,\}\big|, \tag{3}$$

so that the cost of a point query is $\sum_l \omega_l\, c_r$. A node is visited only if its parent was, which invites the conclusion that overlap compounds down the tree. It does not. Measurement (Section 8) shows that ω_l reaches a value characteristic of the construction method within one or two levels of the root and stays there, so that

$$P_{\text{point}} \approx \omega_\infty H c_r. \tag{4}$$

The quality of construction thus enters the cost of a search as a constant factor at every step of the descent, and the correct basis for comparing construction methods is $P_{\text{build}} + nP_{\text{point}}$ for the expected number n of queries over the lifetime of the index — not P_{build} alone, which is what such comparisons customarily report.

5. Sorted construction at framework level

5.1. The support function

We extend the operator class with one optional support function returning a comparator over keys. If it is present, the index is built by sorting all entries with that comparator, filling leaf pages consecutively up to the fill factor, forming an entry for the level above from `union` of each completed page, and repeating level by level until a single page remains.

The order must be declared by the operator class and not passed as a parameter of the index. In an early version of the implementation the comparator was supplied as a relation option, which allowed any function of matching signature to be named — that is, allowed a silently incorrect index to be built [29]. Moving the declaration into the operator class makes the order part of the definition of the index type and allows it to be validated at index creation.

5.2. Locality-preserving orders

For a point type we use the Z-order, or Morton code [30, 31]: coordinates are reinterpreted as unsigned integers and their bits interleaved. Points close in space receive close Z values; the converse does not hold, but for construction this affects the quality of the tree, not its correctness.

Two implementation details are worth recording. The bit pattern of an IEEE 754 value read as an integer is order-preserving for non-negative values and reversed for negative ones; for preserving locality this suffices, since the discontinuity occurs only at zero. Not-a-number values are unordered, and their placement in the order is not neutral: it must agree with the behaviour of `union` and `consistent`, which requires an explicit convention.

That the order is a property of the operator class and nothing else is demonstrated in Section 8 by supplying the Hilbert curve as a drop-in alternative: an operator class differing from the built-in one in exactly one support function out of ten, with no change to the tree, to logging or to the concurrency protocol.

5.3. Bottom-up packing and right links

Constructing bottom-up, the contents of a page are known before it is written, so `union` is applied once per page rather than incrementally.

There is no concurrency during construction: no search and no split can occur. Formally the right links of Kornacker's protocol therefore need not be set. We set them nevertheless. The completeness of the right-link chain at every level is a structural invariant that index verification tools check; if a sorted build left it unsatisfied, a verification tool would have to distinguish indexes by the way they were built. We state this as a design rule: an optimisation must not reduce the set of checkable properties of a structure.

6. Non-leaf key overlap

6.1. Why occupancy-based packing fails

Figure 1 shows the mechanism. The procedure of Section 5 fills a page to the fill factor and starts a new one, that is, chooses split points by a single criterion — occupancy. In one dimension that criterion is optimal. In more than one it is not: an order preserves locality only approximately, and a page boundary falling inside a dense region produces two neighbouring pages whose keys overlap substantially.

The effect is large and it is not confined to the level at which it arises. Measured per level on ten million uniform points, occupancy-based packing settles at $\omega_\infty \approx 4$ — a point query enters four subtrees at every level instead of one.

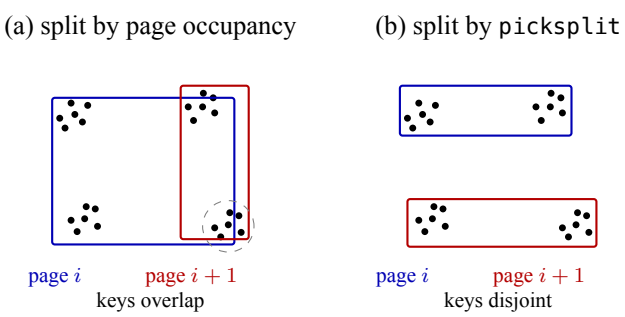

Figure 1: The same entries under the same order; only the choice of page boundaries differs. Filling each page to capacity (a) puts the boundary in the middle of a dense region, so the bounding keys of two neighbouring pages cover almost the same area and a query matching either enters both subtrees. Delegating the choice to the operator class (b) lets boundaries follow the data and the keys come out disjoint. Rectangles are the keys stored in the parent.

6.2. Delegating split selection to the operator class

Rather than inventing a criterion for choosing split points, we use the one the operator class already provides. Entries of a level are accumulated in a buffer of k pages; `picksplit` is applied to the buffer, recursively, until every part fits on a page; the parts are written as pages of the level.

The buffer must be large enough that the partition algorithm can choose a cut that does not coincide with a page boundary, and small enough that partitioning stays cheap. Section 8 measures the dependence on k and shows that the value shipped in PostgreSQL, $k = 4$, is conservative.

Note that this uses `picksplit` in a mode it was not designed for — partitioning a set larger than one page rather than an overflowing node — which is why the recursion is necessary, and which has a consequence that must be stated: a split algorithm of super-linear complexity degrades when handed k times as much data at once. For operator classes with such an algorithm we do not enable sorted construction; improving their `picksplit` is a prerequisite.

7. Implementation

The design is implemented in PostgreSQL and shipped in three releases: sorted construction and the sort support interface of the operator class in release 14 (commit `16fa9b2b30a`), with validation of the support function (`6f0bc5e1daf`) and right links set during construction (`265ea567852`); split selection by `picksplit` over a buffer in release 15 (`f1ea98a7975`); sorted construction for the `btree_gist`

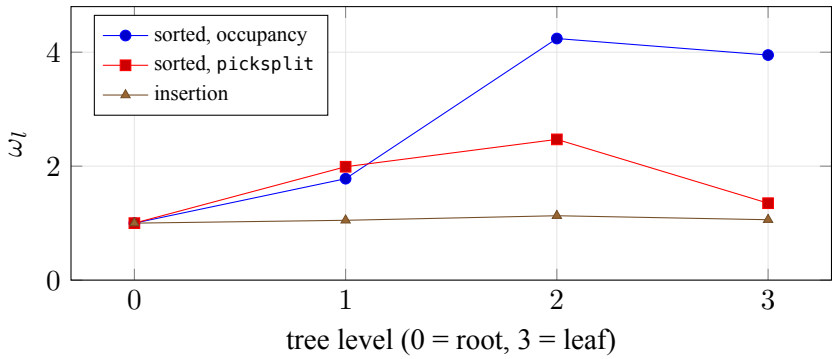

Figure 2: Overlap per level, $N = 10^7$ uniform points, 2000 probes. Each method reaches a value characteristic of it within one or two levels of the root and holds it; overlap does not compound with depth. The upper levels are bounded by the number of nodes on them.

operator classes, made the default strategy, in release 18 (`e4309f73f69`). The discussions that led to the design, including alternatives that were rejected, are publicly archived [29, 32].

Being part of the standard distribution, the implementation is available to every user of the system without additional steps, and every measurement below is reproducible on public releases.

8. Evaluation

8.1. Setup

All measurements were taken on a 16-core AMD EPYC machine with 31 GiB of RAM, `shared_buffers` set to 8 GB and `maintenance_work_mem` to 1 GB. Configurations differ only by the change under study; where a change is a commit, the baseline is its immediate parent, built from the same tree.

The primary metric is the *number of index page accesses per query*, obtained from `EXPLAIN (ANALYZE, BUFFERS)`. It equals $\sum_l \omega_l$ of Section 4, it is independent of the storage device, and it is reproducible on other hardware. Wall-clock times are reported for construction, where they are meaningful, and are stated with the caveats they deserve.

Query windows are scaled so that the expected number of matching rows does not depend on the size of the dataset: $d = 0.5/\sqrt{N}$ for uniform data. For real data the window was calibrated by measurement (Section 8.8); a window chosen by absolute size gives a hundred matching rows in dense regions and measures selectivity rather than tree quality.

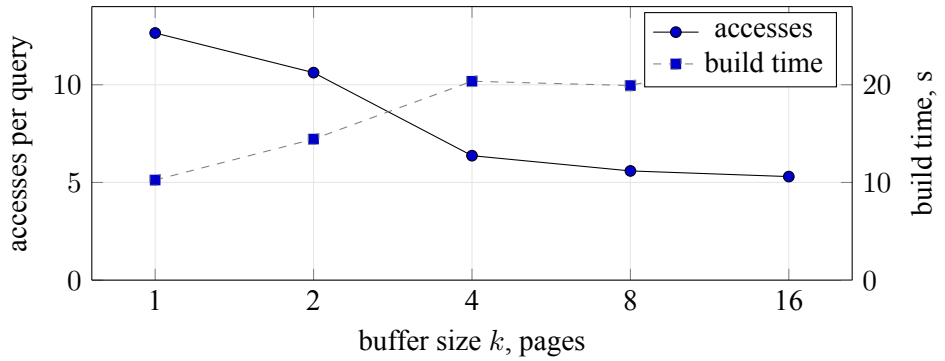

Figure 3: Dependence on the buffer size, $N = 10^7$ uniform points. $k = 1$ reproduces occupancy-based packing. The value shipped in PostgreSQL, $k = 4$, is conservative: $k = 8$ improves query cost by 12 % at no cost in build time.

Table 1: Overlap per level, $N = 10^7$, 2000 probes

	insertion		occupancy		`picksplit`	
Level	nodes	ω_l	nodes	ω_l	nodes	ω_l
0 (root)	1	1.00	1	1.00	1	1.00
1	8	1.05	3	1.78	5	1.99
2	823	1.13	363	4.24	601	2.47
3 (leaf)	90 076	1.06	60 241	3.95	80 737	1.35

8.2. Reconstructing per-level overlap

Internal keys cannot be read directly: the page inspection extension decodes index tuples with the tuple descriptor of the index, whose attribute type is the column type rather than the storage type of the operator class, so that for `point_ops` a `box` is printed as a `point` and one of its corners is silently lost. We therefore reconstruct keys bottom-up: the key of a leaf page is the bounding box of its entries, the key of an internal node is the union of its children's keys, and the topology is taken from the downlink pointers, which the decoding defect does not affect. For a sorted build the stored key equals this union exactly, so the reconstruction is not an approximation.

8.3. Overlap does not compound with depth

Table 1 and Figure 2 give ω_l per level for ten million uniform points.

Each method converges to a value characteristic of it within one or two levels of the root and holds it: $\omega_\infty \approx 4.0$ for occupancy-based packing, ≈ 1.3 for

Table 2: Construction method, uniform data

N	construction	build, s	pages	accesses/query
10^6	insertion	4.38	9 115	3.95
10^6	sorted, occupancy	0.64	6 063	10.01
10^6	sorted, `picksplit`	1.48	8 165	4.62
10^7	insertion	69.61	90 908	5.19
10^7	sorted, occupancy	8.65	60 608	12.23
10^7	sorted, `picksplit`	19.80	81 277	6.10

Table 3: Accesses per query by cluster size, $N = 10^6$

c	occupancy	`picksplit`	ratio
uniform	9.92	4.69	2.12
0.5	9.65	3.48	2.77
1	8.19	3.22	2.54
2	7.78	2.88	2.70
4	6.33	2.97	2.13
8	6.36	2.52	2.52

`picksplit`-based partitioning, ≈ 1.05 for insertion. The upper levels are an exception only because ω_l there is bounded by the number of nodes. This is the empirical content of (4), and it explains why the ratio between methods stays constant as the tree grows deeper.

8.4. Uniform data

Relative to insertion, occupancy-based packing builds 6.8 and 7.1 times faster and makes queries 2.53 and 2.36 times more expensive. `picksplit`-based partitioning builds 3.0 and 3.8 times faster and stays within 17 and 18 % of insertion on query cost: it removes 89 and 87 % of the penalty. The residual is the price of choosing split points within a buffer rather than globally over a level.

Both sorted variants produce a *smaller* index than insertion — by 33 and 11 % respectively — because splits leave pages about 70 % full while consecutive packing fills them to the configured factor. The common expectation that better tree quality costs space does not hold here.

Table 3 varies the degree of clustering: entries are drawn from $N/(cf)$ dense clusters of cf points each, where f is the measured leaf fan-out, so that c is the size of a cluster in units of page capacity.

Table 4: Occupancy packing at reduced fill factor against `picksplit`, $N = 10^7$

Split rule	fill factor	pages	accesses/query
`picksplit`	default	81 326	**6.86**
occupancy	default	60 608	12.67
occupancy	80	68 496	13.15
occupancy	70	78 127	13.95
occupancy	**65**	**84 036**	**13.64**
occupancy	60	91 746	13.72
occupancy	50	109 892	14.49

The ratio stays in the range 2.1–2.8 throughout. We had expected a resonance: the penalty should be largest when a cluster does not fit a whole number of pages, so that boundaries systematically fall inside clusters, and smallest when c is integral. No such dependence appears. Clustered data does punish occupancy-based packing somewhat harder than uniform data (2.5–2.8 against 2.12), which is consistent with the mechanism of Figure 1, but the effect is not a function of c in the way the mechanism suggests. Both build time and index size are independent of c to within one per cent.

8.5. Is it the split rule, or merely the page count?

In every measurement above, query cost falls as the number of index pages rises, which admits a rival explanation: perhaps the benefit comes not from where the split points are placed but from the tree being less bushy, with fewer entries per page to examine. The classical estimate for a search tree in which all children of a visited node must be examined puts the optimal branching factor at about e, which makes the rival explanation plausible a priori. It also has a cheap remedy that would make this paper unnecessary: lower the fill factor.

Table 4 tests it. Data and probes are generated from a fixed seed, so both builds see the same ten million points and the same two thousand queries.

At an essentially equal page count—84 036 against 81 326—occupancy packing costs 13.64 accesses per query against 6.86, twice as many. Lowering the fill factor does not help at all; it mildly hurts, the cost drifting from 12.67 to 14.49 as the index grows by 80 %. The rival explanation is refuted: what the operator class contributes is the placement of the boundaries, not the number of them.

Two further remarks. First, the branching-factor argument holds only under unbounded memory: a page is the unit of exchange with the buffer cache, and a less bushy tree occupies more pages, fits it worse, and loses on data that does not fit in memory—which on the real dataset of Section 8.8 is the operating regime.

Table 5: Accesses per query: order × split rule

	10^6 uniform		1.38×10^8 OSM	
Split rule	Z-order	Hilbert	Z-order	Hilbert
occupancy	10.01	5.57	15.30	10.17
`picksplit`	4.62	4.11	8.30	8.27

Table 6: Dependence on buffer size k, $N = 10^7$ uniform

k	build, s	pages	accesses/query
1	8.89	54 351	12.65
2	13.05	81 593	10.62
4	15.71	81 287	6.37
8	19.99	81 682	5.59
16	23.35	81 650	5.30

Second, `picksplit`-based construction produced 81 326 pages at both the default fill factor and at 65, that is, it does not honour the parameter; we return to this in Section 9.

8.6. Order and split points are separate levers

Table 5 crosses the two.

On uniform data the two are partially interchangeable: the curve recovers 44 % of the penalty of occupancy-based packing, the split rule recovers 53 %, and together they come within 4 % of insertion. On real data the relation is asymmetric: the curve recovers 33 % while split points are chosen by occupancy, and nothing at all once they are chosen by the operator class. An elaborate space-filling order therefore pays off only while the partitioning rule stays naive — a conclusion that matters to whoever writes an operator class, and one that would have been invisible on synthetic data alone.

Build times for the Hilbert variant are not comparable with Z-order: the experimental comparator used here does not implement abbreviated keys, so the difference measures the presence of a sorting optimisation rather than a property of the curve.

8.7. Buffer size

Table 6 and Figure 3 give the dependence.

Table 7: OSM, $N = 1.38 \times 10^8$; 5000 selective and 500 non-selective probes

construction	build, s	pages	selective	non-selective
insertion	1 160.3	1 285 510	16.73	40.95
sorted, occupancy	322.7	836 842	15.30	61.01
sorted, `picksplit`	396.3	1 128 374	8.30	48.23

$k = 1$ reproduces occupancy-based packing, as it must: there is nothing to choose within a single page. The parameter is thus continuous between the two strategies rather than a switch between them. The value shipped in PostgreSQL, $k = 4$, sits close to the knee: $k = 8$ improves query cost by 12 % for 27 % more build time, and $k = 16$ by 17 % for 49 % more. Every step is a trade rather than a free improvement, which is visible only in medians of repeated runs: single build times at this scale vary by tens of percent (Section 10). Index size is independent of k from $k = 2$ on. Returns diminish sharply after $k = 4$, which is consistent with the role of the buffer: it exists so that a cut need not coincide with a page boundary, and by $k = 8$ there is enough freedom for that.

8.8. Real data

We use 138 078 566 node coordinates from the OpenStreetMap extract of the Netherlands. The index occupies 6.5–8.8 GB against 8 GB of buffer cache, so unlike every preceding measurement it does not fit in memory.

Node density varies by orders of magnitude, and probes drawn from the data land in cities. Calibration: a window of 5×10^{-5} degrees returns 2.7 rows on average, one of 5×10^{-4} returns 112. We report both — the first as a point query, the second as a deliberately non-selective one.

Three observations.

The ratio between the two sorted variants, 1.84, agrees with the 2.0–2.2 measured on uniform data: the central result transfers to real data unchanged.

On non-selective queries the ratio falls to 1.26. When a query returns hundreds of rows its cost is dominated by reading them rather than by navigation. This is a boundary of applicability worth stating plainly: the construction method matters for selective queries and is nearly irrelevant for scanning large regions.

The cost of good partitioning falls with scale but does not vanish: `picksplit` costs 87 % more than occupancy-based packing at 10^7 and 23 % more at 1.38×10^8, as I/O takes over an increasing share of build time.

8.9. An unexpected result

On this dataset *insertion is no better than occupancy-based packing*: 16.73 accesses per selective query against 15.30, while taking three times as long to build as `picksplit` and producing the largest index. Against `picksplit`, at 8.30, it is twice as expensive. On uniform data the ordering was the opposite, with insertion nearly optimal (Table 1).

The margin over occupancy-based packing is 9 %, and we are deliberately exact about how little that carries. It is close to the threshold below which we treat differences as not established, and it is sensitive to the choice of probes: measured against independently drawn probe sets rather than the single fixed set used here, the same comparison ranges over a factor of two (Section 10). What survives is not a magnitude but a reversal — on skewed data the method with global information is at least as good as local insertion, whereas on uniform data insertion wins comfortably.

The plausible explanation is that on strongly skewed data the decisions taken during insertion — subtree choice and node split — are local, and their errors accumulate; this is the classical weakness of the R-tree that the R*-tree was designed to address [17]. Sorted construction has global information and is free of it. On uniform data there is nothing to accumulate, which is why the effect does not appear there.

We report this as an observation requiring confirmation rather than as a result: it rests on a single dataset, and its magnitude already shrank once under a more careful measurement. If it holds, it changes the standing of sorted construction from a trade of quality for speed into an improvement on both axes for skewed data — that is, for the practically interesting case.

8.10. Where the CPU goes

Table 8 profiles a backend performing a spatial join of 200 000 outer rows against the index, some two million result rows, with index and table resident in the buffer cache so that no I/O is involved.

Work on entries of an index page accounts for 46.8 % in total, buffer handling for 18.4 % and the table side for 15.6 %.

Two consequences. First, the mechanics of iterating over entries cost 1.6 times more than evaluating the predicate, so the promising direction for CPU work is to examine fewer entries rather than to make each examination cheaper. Second, buffer lookup and pinning together account for 18.4 % and are proportional to the number of pages visited, that is to ω_∞: the contribution of this paper, argued so far in terms of I/O, yields a comparable saving in CPU on data that fits in memory.

Table 8: Backend CPU during descent, self time

Share	Function	What it is
22.4 %	`gistScanPage`	loop over entries of a page
14.1 %	`gist_point_consistent`	the predicate itself
12.8 %	`hash_search`	buffer lookup table
10.2 %	`heap_page_prune_opt`	table side
6.7 %	`MemoryContextReset`	
5.7 %	`PinBuffer`	pinning pages
5.4 %	`heap_hot_search_buffer`	table side
4.7 %	`FunctionCall5Coll`	invoking the predicate
3.3 %	`gistcheckpage`	
2.3 %	`gistdentryinit`	preparing an entry

9. Deployment experience

The design has shipped since release 14 (2021) and has been the default strategy for the `btree_gist` operator classes since release 18 (2025). Over that period one substantive failure occurred, and it is instructive.

The first attempt to extend sorted construction to `btree_gist` was reverted on the day it was committed. Two defects were involved: one in the regression test environment, and one substantive — the comparator for the bit-string type used a comparison that does not agree with the semantics of the type. Work resumed four years later with an implementation written from scratch [32].

The lesson is about framework design rather than about that patch. Consistency of the comparator with `union` and `consistent` is a property of the operator class that cannot be established by testing the build: an inconsistent comparator yields an index that looks well-formed and silently loses part of the answers. It is checkable only as a structural invariant of the produced tree, namely that every parent key covers the keys of its children. A framework that introduces a new support function is thereby obliged to introduce the check of the corresponding invariant as well; we did not do so at the time, and the omission cost four years.

10. Threats to validity

Reconstruction of internal keys.. Per-level overlap rests on keys recovered bottom-up rather than read from the page. The recovery is exact for a sorted build, where the stored key equals the union of the children's keys by construction, and it is exact for the insertion-built index only if no key has been left wider than that union. Deletions

can leave such keys behind; our datasets are insert-only, so the concern does not arise here, but a measurement over a workload with deletions would need the keys read directly. That in turn requires fixing the decoding of internal keys in the page inspection extension, which currently interprets them with the tuple descriptor of the index rather than the storage type of the operator class and silently drops part of a key whenever the two differ.

Counters and times are not equally trustworthy.. The two must be reported separately, and conflating them is easy. Accesses per query, measured with both the data and the probe set fixed by seed, are deterministic: they repeat to within a fraction of a percent, and every page-access figure in Sections 8.4–8.7 carries that guarantee. Build times do not. Repeating the 10^7 configurations three times gave 6.78, 8.46 and 8.63 seconds for one and the same configuration — a spread of 27 %. All times reported here are medians of three sequential runs; we treat time differences below roughly 30 % as not established, and ratios of times only when they exceed that.

Access counts depend on the choice of probes, and more so than times depend on the machine.. On skewed data this is the larger threat of the two, and it is not visible from a single run because the counter is perfectly repeatable for a given probe set. One configuration on the OSM data gave 23.66 and 16.83 accesses per query for two independently drawn sets of 5000 probes. All real-data figures reported here use one fixed set of 20 000 probes, generated once and shared by every index compared. Fixing the data by seed is not enough — the queries must be fixed too, and on skewed data this matters more than anything about the machine.

Window calibration on skewed data.. An early version of the real-data experiment fixed the query window by absolute size. Because node density in OpenStreetMap varies by orders of magnitude and probes are drawn from the data, the window returned 112 rows on average instead of the intended one, and the difference between construction methods shrank from a factor of two to 23 %. The reported measurements calibrate the window by the expected number of matching rows. Any comparison of index quality on skewed data is worthless without this calibration, which is easy to overlook.

One operator class.. All quantitative results concern points, under either of two orders. The claim that the technique is framework-level rests on the interface argument and on the fact that a second order was added without touching the framework, not on measurements over several key types. Operator classes over signatures or ranges may behave differently, and we do not know whether they do.

Comparison of curves by build time.. Invalid as measured: the experimental Hilbert comparator does not implement abbreviated keys while the built-in Z-order comparator does. Only the query-side comparison of curves is sound.

11. Limitations

The method requires a locality-preserving order, and for operator classes whose keys admit none it is inapplicable — with no diagnostic short of measuring query quality. It requires a split algorithm that does not degrade on k pages of input. Its correctness rests on the consistency of the comparator with the rest of the operator class, which the framework does not enforce. The gain does not extend to subsequent maintenance: entries inserted after the build follow the ordinary insertion path.

The measurements are confined to one operator class for points and its Hilbert variant, to two synthetic distributions and one real dataset, and to a single run per configuration. The comparison of curves by build time is invalid until abbreviated keys are implemented for the Hilbert comparator. The result of Section 8.9 requires confirmation.

12. Conclusion

Bulk loading of an extensible index framework reduces to sorting given one additional support function, and the order it declares must come from the operator class because nothing else knows it. The reduction alone is not a usable result: choosing page boundaries by occupancy raises the steady-state overlap from about 1.3 to about 4, and the index is built several times faster and answers selective queries twice as expensively. Delegating the choice of split points back to the operator class, over a buffer of several pages, removes 87–89 % of that penalty. Both parts are implemented in PostgreSQL and shipped to its users.

The order and the split rule are separate levers with an asymmetric interaction: on real data an elaborate curve helps only while the split rule is naive. The buffer size shipped in the implementation is conservative by 12 %. And the question of whether sorted construction is merely faster than insertion, or actually better on skewed data, is open and worth settling.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used Claude (Anthropic) for the following: drafting the text of the manuscript; implementing the measurement

scripts and the experimental Hilbert-order comparator described in Section 8; running the experiments and tabulating their results; producing the figures; and searching for and assembling the bibliography. The algorithms described in this paper, their implementation in PostgreSQL, and the deployment experience reported in Section 9 predate and are independent of that use. All measurements were rechecked by the author against the recorded scripts and outputs, which are available with the artifact. After using this tool the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

Declaration of competing interest

The author declares no competing interests.

References

[1] J. M. Hellerstein, J. F. Naughton, A. Pfeffer, Generalized search trees for database systems, University of Wisconsin-Madison, 1995.

[2] M. Kornacker, C. Mohan, J. M. Hellerstein, Concurrency and recovery in generalized search trees, in: ACM SIGMOD Record, Vol. 26, ACM, 1997, pp. 62–72.

[3] A. Guttman, R-trees: a dynamic index structure for spatial searching, in: ACM SIGMOD Record, Vol. 14, ACM, 1984, pp. 47–57.

[4] I. Chilingarian, O. Bartunov, J. Richter, T. Sigaev, Postgresql: the suitable dbms solution for astronomy and astrophysics, in: Astronomical Data Analysis Software and Systems (ADASS) XIII, Vol. 314, 2004, p. 225.

[5] Рогов Е. В., PostgreSQL 18 изнутри, ДМК Пресс, М., 2026.

[6] N. Roussopoulos, D. Leifker, Direct spatial search on pictorial databases using packed r-trees, in: ACM SIGMOD Record, Vol. 14, ACM, 1985, pp. 17–31.

[7] I. Kamel, C. Faloutsos, On packing r-trees, in: Proceedings of the Second International Conference on Information and Knowledge Management, ACM, 1993, pp. 490–499.

[8] S. T. Leutenegger, M. A. Lopez, J. Edgington, Str: a simple and efficient algorithm for r-tree packing, in: Proceedings 13th International Conference on Data Engineering, IEEE, 1997, pp. 497–506.

[9] Y. J. García, M. A. López, S. T. Leutenegger, A greedy algorithm for bulk loading r-trees, in: Proceedings of the 6th ACM International Symposium on Advances in Geographic Information Systems, ACM, 1998.

[10] C. Böhm, H.-P. Kriegel, Efficient bulk loading of large high-dimensional indexes, in: Data Warehousing and Knowledge Discovery (DaWaK), 1999, pp. 251–260.

[11] D. Achakeev, B. Seeger, P. Widmayer, Sort-based query-adaptive loading of r-trees, in: Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM), ACM, 2012.

[12] D. Achakeev, B. Seeger, Sort-based parallel loading of r-trees, in: Proceedings of the 1st ACM SIGSPATIAL International Workshop on Analytics for Big Geospatial Data, ACM, 2012.

[13] J. Qi, et al., PLATON: Top-down R-tree packing with learned partition policy, Proceedings of the ACM on Management of Data (2023). doi:10.1145/3626742.

[14] J. van den Bercken, B. Seeger, P. Widmayer, A generic approach to bulk loading multidimensional index structures, in: Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB), 1997, pp. 406–415.

[15] L. Arge, K. H. Hinrichs, J. Vahrenhold, J. S. Vitter, Efficient bulk operations on dynamic r-trees, Algorithmica 33 (1) (2002) 104–128.

[16] L. Arge, M. de Berg, H. J. Haverkort, K. Yi, The priority r-tree: a practically efficient and worst-case optimal r-tree, in: Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data, ACM, 2004, pp. 347–358.

[17] N. Beckmann, H.-P. Kriegel, R. Schneider, B. Seeger, The r*-tree: an efficient and robust access method for points and rectangles, in: ACM SIGMOD Record, Vol. 19, ACM, 1990, pp. 322–331.

[18] N. Beckmann, B. Seeger, A revised r*-tree in comparison with related index structures, in: Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data, ACM, 2009, pp. 799–812.

[19] A. Borodin, S. Mirvoda, S. Porshnev, Improving penalty function of r-tree over generalized index search tree, in: IEEE 2nd International Conference on Big Data Analysis (ICBDA), IEEE, 2017.

[20] M. Y. Eltabakh, R. Eltarras, W. G. Aref, Space-partitioning trees in postgresql: realization and performance, in: Proceedings of the 22nd International Conference on Data Engineering (ICDE), IEEE, 2006.

[21] V. Gaede, O. Günther, Multidimensional access methods, ACM Computing Surveys 30 (2) (1998) 170–231. doi:10.1145/280277.280279.

[22] G. Graefe, Modern b-tree techniques, Foundations and Trends in Databases 3 (4) (2011) 203–402. doi:10.1561/1900000028.

[23] G. Graefe, More modern b-tree techniques, Foundations and Trends in Databases 13 (3) (2024) 169–249. doi:10.1561/1900000070.

[24] D. Comer, The ubiquitous b-tree, ACM Computing Surveys 11 (2) (1979) 121–137.

[25] R. Bayer, E. McCreight, Organization and maintenance of large ordered indexes, Acta Informatica 1 (3) (1972) 173–189.

[26] A. Borodin, S. Mirvoda, I. Kulikov, S. Porshnev, Optimization of memory operations in generalized search trees of postgresql, in: Beyond Databases, Architectures and Structures (BDAS), Communications in Computer and Information Science, Springer, 2017, pp. 224–232.

[27] A. Borodin, S. Mirvoda, S. Porshnev, Intra-page indexing in generalized search trees of postgresql, in: Data Analytics and Management in Data Intensive Domains (DAMDID/RCDL 2019), Kazan, Russia, Vol. 2523 of CEUR Workshop Proceedings, 2019.
URL https://ceur-ws.org/Vol-2523/paper17.pdf

[28] A. Aggarwal, J. S. Vitter, The input/output complexity of sorting and related problems, Communications of the ACM 31 (1988) 1116–1127.

[29] A. Borodin, et al., Yet another fast gist build, Discussion on the pgsql-hackers mailing list (2019).
URL https://postgr.es/m/1A36620E-CAD8-4267-9067-FB31385E7C0D@yandex-team.ru

[30] G. M. Morton, A computer oriented geodetic data base and a new technique in file sequencing, Tech. rep., IBM Ltd., Ottawa, Canada (1966).

[31] J. A. Orenstein, T. H. Merrett, A class of data structures for associative searching, Proceedings of the 3rd ACM SIGACT-SIGMOD Symposium on Principles of Database Systems (1984) 181–190.

[32] B. Helmle, A. Borodin, et al., Sorted gist index builds for btree_gist, Discussion on the pgsql-hackers mailing list (2024).
URL https://postgr.es/m/64d324ce2a6d535d3f0f3baeeea7b25beff82ce4.camel@oopsware.de